AIAC ASSIGNMENT – 19.2

Name : G.Dhanush Reddy

H.No : 2303A51619

Batch : 22

Task 1 – Python to Java Conversion

Task:

Convert a Python program that performs file handling operations into an equivalent Java program using AI assistance.

Code:

Python:

Welcome | 19.2.py | 19.java | input.txt | output.t

19.2.py > ...

```python
'''Task 1 - Python to Java Conversion
Task:
Convert a Python program that performs file handling operations into an
equivalent Java program using AI assistance.
Instructions:
• Prompt AI to translate a Python script that reads from and writes
to a file into Java.
• Ensure proper handling of file streams, exceptions, and
input/output formatting.
• Verify that the Java program produces the same file content and
console output as the Python version.
• Test both implementations with sample input files.'''
# Python version of the file handling program
# take from input.txt and write to output.txt
def read_and_write_file(input_file, output_file):
    try:
        with open(input_file, 'r') as infile:
            content = infile.read()
            print("Content read from file:")
            print(content)

        with open(output_file, 'w') as outfile:
            outfile.write(content)
            print(f"Content written to {output_file}")

    except FileNotFoundError:
        print(f"Error: The file {input_file} was not found.")
    except IOError as e:
        print(f"IO error occurred: {e}")
# Example usage
input_file = 'input.txt'
output_file = 'output.txt'
read_and_write_file(input_file, output_file)
```

Java:

```java
19.java > ...
/*
read the code from 19.py and convert that code to java which reads from input.txt and writes to output.txt
*/
import java.io.*;
import java.nio.file.Files;
import java.nio.file.Paths;

class Main {
    Run main | Debug main
    public static void main(String[] args) {
        try {
            String inputFile = "input.txt";
            String outputFile = "output.txt";

            // Read entire content from input file
            String content = new String(Files.readAllBytes(Paths.get(inputFile)));

            // Print content to console
            System.out.println("Content read from file:");
            System.out.println(content);

            // Write content to output file
            Files.write(Paths.get(outputFile), content.getBytes());
            System.out.println("Content written to " + outputFile);

        } catch (java.nio.file.NoSuchFileException e) {
            System.out.println("Error: The file was not found.");
        } catch (IOException e) {
            System.out.println("IO error occurred: " + e.getMessage());
        }
    }
}
```

Output:

```
output.txt
hi this is pikachuu!!

```

```
Listening on 51765
User program running
User program finished
Content read from file:
hi this is pikachuu!!

Content written to output.txt
```

Observation:

The AI assistant makes converting code between languages much easier. Instead of manually rewriting the Python file-handling program into Java, the AI generates equivalent Java code with proper handling of file streams, exceptions, and input/output formatting. This saves time and effort, ensures accuracy. By testing with sample files, it becomes clear that the AI not only translates syntax but also preserves the logic, making cross-language coding tasks more straightforward and reliable.

Task 2 – C++ to Python Function Conversion

Task:

Translate a C++ program that implements a sorting algorithm into Python using AI assistance.

Instructions:

• Provide AI with a C++ implementation of a sorting technique (e.g., Bubble Sort or Selection Sort).

• Ask AI to generate an equivalent Python implementation.

• Ensure logical correctness and validate results using multiple test cases.

• Confirm that the Python version maintains the same sorting behavior.

Code :

Cpp:

19.2.cpp

```
/*Task 2 – C++ to Python Function Conversion
Task:
Translate a C++ program that implements a sorting algorithm into
Python using AI assistance.
Instructions:
• Provide AI with a C++ implementation of a sorting technique
(e.g., Bubble Sort or Selection Sort).
• Ask AI to generate an equivalent Python implementation.
• Ensure logical correctness and validate results using multiple
test cases.
• Confirm that the Python version maintains the same sorting
behavior.
*/
Here is a C++ implementation of the Bubble Sort algorithm:
```cpp
#include <iostream>
#include <vector>
using namespace std;
void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
            }
        }
    }
}
int main() {
    vector<int> arr = {64, 34, 25, 12, 22, 11, 90};
    bubbleSort(arr);
    cout << "Sorted array: ";
    for (int i = 0; i < arr.size(); i++) {
        cout << arr[i] << " ";
    }
    return 0;
}

```

Python:
```

19.2.py > ...

```python
'''Translate a C++ program that implements a sorting algorithm into
Python using AI assistance.
Instructions:
• Provide AI with a C++ implementation of a sorting technique
(e.g., Bubble Sort or Selection Sort).
• Ask AI to generate an equivalent Python implementation.
• Ensure logical correctness and validate results using multiple
test cases.
• Confirm that the Python version maintains the same sorting
behavior.'''
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
# Example usage
if __name__ == "__main__":
    test_cases = [
        [64, 34, 25, 12, 22, 11, 90],
        [5, 1, 4, 2, 8],
        [38, 27, 43, 3, 9, 82, 10]
    ]
    for i, test in enumerate(test_cases):
        sorted_arr = bubble_sort(test)
        print(f"Test case {i+1}: {sorted_arr}")
```

Observation :

The AI assistant makes converting algorithms between languages very convenient. By providing a C++ sorting program, the AI quickly generates an equivalent Python version that preserves the same logic and behavior. This saves time compared to rewriting code manually and ensures accuracy across different test cases. The AI not only translates syntax but also maintains the algorithm’s correctness, making it easier to validate results and understand how the same sorting technique works in both languages.

Task 3 – SQL to Pandas Query

Task:

Convert a SQL query involving GROUP BY and aggregate functions

into an equivalent Pandas Data Frame operation.

Code :

```python
'''Task 3 - SQL to Pandas Query
Task:
Convert a SQL query involving GROUP BY and aggregate functions
into an equivalent Pandas Data Frame operation.
Instructions:
• Provide AI with a SQL query that calculates aggregate values
(e.g., total sales per department).
• Prompt AI to generate equivalent Pandas code using groupby()
and aggregation functions.
• Test the generated code using a sample dataset.
• Verify that both SQL and Pandas outputs match.'''
import pandas as pd
# Sample dataset
data = {
    'Department': ['Sales', 'Sales', 'HR', 'HR', 'IT', 'IT'],
    'Employee': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank'],
    'Sales': [100, 150, 200, 250, 300, 350]
}
df = pd.DataFrame(data)
# SQL query equivalent: SELECT Department, SUM(Sales) FROM table GROUP BY Department
grouped_df = df.groupby('Department')['Sales'].sum().reset_index()
print(grouped_df)
```

Output:

```
  Department  Sales
0         HR    450
1         IT    650
2      Sales    250
PS C:\Users\Dhanush Reddy\3RD Year\AIAC\19>
```

Observation :

The AI assistant makes it easy to move from SQL to Pandas by generating equivalent Python code using groupby() and aggregation functions. Instead of manually rewriting queries, the AI quickly translates SQL logic into Pandas operations that produce the same results. This saves time, ensures accuracy, and helps in understanding how database-style queries can be performed directly on data frames. Testing with sample datasets confirms that both SQL and Pandas outputs match, showing how AI simplifies cross-platform data analysis.

Task 4 – Real-Time Application: Multi-Language Snippet

Converter

Scenario:

Students will select a practical real-world application requiring the same logic in two different programming languages.

Example:

• Converting a REST API implementation from Python (Flask) to Java (Spring Boot).

• Translating a data processing script from Python to C++ for performance optimization.

Code:

python.py > ...

```python
# Calculate total and average marks

students = [
    {"name": "A", "marks": [80, 90, 85]},
    {"name": "B", "marks": [70, 60, 75]},
    {"name": "C", "marks": [88, 92, 95]}
]

for student in students:
    total = sum(student["marks"])
    avg = total / len(student["marks"])

    print(f"{student['name']} -> Total: {total}, Average: {avg:.2f}")
```

```cpp
#include <iostream>
using namespace std;
void displayMenu() {
    cout << "Simple Calculator" << endl;
    cout << "1. Addition" << endl;
    cout << "2. Subtraction" << endl;
    cout << "3. Multiplication" << endl;
    cout << "4. Division" << endl;
    cout << "5. Exit" << endl;
}
int main() {
    int choice;
    double num1, num2;

    do {
        displayMenu();
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice >= 1 && choice <= 4) {
            cout << "Enter two numbers: ";
            cin >> num1 >> num2;
        }

        switch (choice) {
            case 1:
                cout << "Result: " << num1 + num2 << endl;
                break;
            case 2:
                cout << "Result: " << num1 - num2 << endl;
                break;
            case 3:
                cout << "Result: " << num1 * num2 << endl;
                break;
            case 4:
                if (num2 != 0) {
                    cout << "Result: " << num1 / num2 << endl;
                } else {
                    cout << "Error: Division by zero is not allowed." << endl;
                }
                break;
            case 5:
                cout << "Exiting the calculator. Goodbye!" << endl;
                break;
            default:
                cout << "Invalid choice. Please try again." << endl;
        }
```

Observation :

The AI assistant makes real-world multi-language conversions much more practical. For example, when converting a REST API from Python (Flask) to Java (Spring Boot) or translating a data processing script from Python to C++ for performance, the AI generates equivalent code that preserves the same logic while adapting to each language's syntax and framework. This saves a lot of effort compared to rewriting everything manually and ensures consistency across implementations. It shows how AI can bridge the gap between different programming environments, making cross-language development faster, easier, and more reliable.

Task -5: (Basic Function Translation (Python to C++)

Translate a simple Python function that checks whether a number is even or odd into C++ using AI assistance.

Code :

python.py > ...

```python

def check_even_odd(num):
    if num % 2 == 0:
        return "Even"
    else:
        return "Odd"
# Example usage
number = 10
result = check_even_odd(number)
print(f"The number {number} is {result}.")
number = 15
result = check_even_odd(number)
print(f"The number {number} is {result}.")

```

```cpp
#include <iostream>
using namespace std;
bool isEven(int number) {
    return number % 2 == 0;
}
int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;

    if (isEven(num)) {
        cout << num << " is even." << endl;
    } else {
        cout << num << " is odd." << endl;
    }

    return 0;
}
```

Output :

```
The number 10 is Even.
The number 15 is Odd.
```

Observation:

The AI assistant makes translating simple functions between languages very easy. By providing a Python function that checks if a number is even or odd, the AI quickly generates an equivalent C++ version with correct syntax, input handling, and output formatting. This saves time compared to writing the code manually and ensures the logic remains consistent across both languages. Testing with different inputs confirms that the AI preserves the same behavior, showing how it simplifies cross-language programming tasks

---